

Corrigé du TD5 — Analyse sémantique

Compilation — Master 1 Informatique (UBO)

À retenir

Synthétisé : calculé à partir des fils (bas → haut). **Hérité** : calculé à partir du père et des frères gauches (haut → bas).

Une DDS s'écrit : pour chaque production, des règles qui calculent les attributs. Notation : $S_0 = S$ en partie gauche, $S_1, S_2, \dots$ = occurrences en partie droite.

Exercice 1

1. Hérité ou synthétisé ?

- **x est hérité** : dans $S \rightarrow aSa$ et $S \rightarrow bSc$, $S_1.x$ est calculé à partir de $S_0.x$ (le père). Pour $S \rightarrow d$, x est utilisé tel quel.
- **y est synthétisé** : $S_0.y$ est calculé à partir de $S_1.y$ (le fils). Pour $S \rightarrow d$, y est calculé à partir de x du même nœud (c'est quand même une règle « de la partie gauche »).

2. Arbre décoré de aabadacaa

Découpage : a (a (b (a d a) c) a) a. Les règles utilisées, de la racine aux feuilles : aSa, aSa, bSc, aSa, d. La racine S' donne $S.x := 0$.

```
S'
└ S (aSa)      x = 0          retourne S.y = 16
  └ S (aSa)    x = 2*0+1 = 1   y = B.y+1 = 15+1 = 16
    └ S (bSc)  x = 2*1+1 = 3   y = C.y+1 = 14+1 = 15
      └ S (aSa) x = 3+5 = 8     y = 3*D.y-10 = 3*8-10 = 14
        └ S (aSa) x = 3+5 = 8   y = E.y+1 = 7+1 = 8
          └ S (d) x = 2*8+1 = 17 y = x-10 = 17-10 = 7
```

Calcul en deux temps : **descente** (x : 0, 1, 3, 8, 17) puis **remontée** (y : 7, 8, 14, 15, 16). Chaque ligne ci-dessus est un nœud S, indenté selon sa profondeur ; x se calcule d'après la ligne du dessus, y d'après la ligne du dessous. **La DDS retourne 16.**

Exercice 2

Langage $(a|b)(ba|aa|ab)^*$: une première lettre, puis des blocs de deux lettres parmi ba, aa, ab (jamais bb). On l'engendre par la grammaire :

```
S -> a P | b P
P -> a a P | b a P | a b P | eps
```

But : le nombre maximum de **a consécutifs** dans le mot.

1. Avec des attributs hérités

Idée : on « descend » deux accumulateurs : c = longueur de la série de a en cours (qui se termine à la position courante) et m = maximum rencontré jusqu'ici. Au bout du mot ($P \rightarrow \text{eps}$), on retourne m.

```
S' -> S (S n'a pas d'attribut hérité)
S -> a P { P.c := 1 ; P.m := 1 }
S -> b P { P.c := 0 ; P.m := 0 }
P -> a a P1 { P1.c := P0.c + 2 ; P1.m := max(P0.m, P0.c + 2) }
P -> b a P1 { P1.c := 1 ; P1.m := max(P0.m, 1) }
P -> a b P1 { P1.c := 0 ; P1.m := max(P0.m, P0.c + 1) }
P -> eps { retourner P.m }
```

- **aa** : la série en cours grandit de 2. **ba** : le b coupe la série, le a ouvre une de longueur 1. **ab** : le a prolonge la série de 1 puis le b la coupe (c repart à 0).
- c et m sont des attributs **hérités** de P (calculés à partir du père).

Exemple — mot baaab = b · aa · ab

$S \rightarrow b P : c=0, m=0$. $P \rightarrow aa P1 : c=2, m=\max(0,2)=2$. $P1 \rightarrow ab P2 : m = \max(2, 2+1) = 3, c=0$. $P2 \rightarrow \text{eps} : \text{retourne } 3$. ✓ (aaa)

2. Avec des attributs synthétisés

Pour un morceau de mot u on synthétise 4 attributs : n (longueur), p (a en tête), s (a en queue), m (série maximale). Un morceau ne contenant que des a vérifie $p = n$. Pour la concaténation de u et v , on pose :

```
conc(u,v).n = u.n + v.n
conc(u,v).p = si u.p = u.n alors u.n + v.p sinon u.p      (u tout en a : la tête continue dans v)
conc(u,v).s = si v.s = v.n alors v.n + u.s sinon v.s      (v tout en a : la queue continue dans u)
conc(u,v).m = max(u.m, v.m, u.s + v.p)                  (série à cheval sur la jonction)
```

Lettres : $A = (n=1, p=1, s=1, m=1)$ pour a ; $B = (1, 0, 0, 0)$ pour b ; epsilon = $(0,0,0,0)$. Les règles :

```
S -> a P    { S.(n,p,s,m) := conc(A, P) }
S -> b P    { S.(n,p,s,m) := conc(B, P) }
P -> a a P1 { P0.(n,p,s,m) := conc(conc(A,A), P1) }
P -> b a P1 { P0.(n,p,s,m) := conc(conc(B,A), P1) }
P -> a b P1 { P0.(n,p,s,m) := conc(conc(A,B), P1) }
P -> eps    { P.(n,p,s,m) := (0,0,0,0) }
Résultat : S'.m = S.m
```

Exemple — mot baaab

$ab \cdot \text{eps} = (n2, p1, s0, m1)$; $aa = (2,2,2,2)$; $aa \cdot (ab) = (4, p=2+1=3, s=0, m=\max(2,1,2+1)=3)$;
 $b \cdot (...) = (5, 0, 0, m=\max(0,3,0+3)=3)$. Résultat : 3. ✓

Remarque : tous les attributs sont calculés à partir des fils, donc la grammaire est S-attribuée (et évaluable pendant une analyse LR).

Exercice 3

1. DDS pour la profondeur maximale (langage de Dyck)

```
S' -> S      { retourner S.d }
S -> ( S1 ) S2 { S0.d := max(S1.d + 1, S2.d) }
S -> eps      { S.d := 0 }
```

d est synthétisé : le contenu entre parenthèses est un niveau plus profond ($S_1.d + 1$), la suite S_2 est au même niveau que S_0 . On prend le plus grand des deux.

2. Arbre décoré pour $((()())()())()$

Le mot a 20 caractères. Il se décompose en $(S1) S2$ avec $S1 = (()())()()()$ et $S2 = \text{eps}$. Chaque nœud S , avec le mot qu'il engendre, sa règle et sa valeur d :

Nœud	Mot engendré	Règle : (S1) S2	d
N0	$((()())()())()$	(N1) eps	$\max(3+1, 0) = 4$
N1	$((()())()())()$	(A) B	$\max(1+1, 3) = 3$
A	$()()$	(eps) A'	$\max(0+1, 1) = 1$
A'	$()$	(eps) eps	$\max(0+1, 0) = 1$
B	$((()())()())$	(C) B'	$\max(2+1, 1) = 3$
C	$()()$	(eps) C'	$\max(0+1, 2) = 2$
C'	$()()$	(E) eps	$\max(1+1, 0) = 2$
E	$()$	(eps) eps	$\max(0+1, 0) = 1$
B'	$()()$	(eps) B''	$\max(0+1, 1) = 1$
B''	$()$	(eps) eps	1
chaque eps	(vide)	$S \rightarrow \text{eps}$	0

La DDS retourne 4 (vérification sur le mot : la profondeur 4 est atteinte à la 12e parenthèse, dans le groupe « $((()(()))$ »).

Exercice 4

Grammaire : $N \rightarrow N B \mid B$; $B \rightarrow 0 \mid 1$. Lire un bit de plus revient à doubler la valeur déjà lue et à ajouter ce bit.

```
S' -> N      { retourner N.v }
N -> N1 B     { N0.v := 2 * N1.v + B.v }
N -> B        { N.v := B.v }
B -> 0        { B.v := 0 }
B -> 1        { B.v := 1 }
```

v est synthétisé (S-attribuée, donc calculable pendant une analyse LR au fil des réductions). L'arbre est penché à gauche ; la valeur de chaque N , du bas vers le haut, pour 1 1 0 1 0 1 0 0 1 :

Bit lu	1	1	0	1	0	1	0	0	1
N.v	1	$2 \cdot 1 + 1 = 3$	$2 \cdot 3 + 0 = 6$	$2 \cdot 6 + 1 = 13$	$2 \cdot 13 + 0 = 26$	$2 \cdot 26 + 1 = 53$	$2 \cdot 53 + 0 = 106$	$2 \cdot 106 + 0 = 212$	$2 \cdot 212 + 1 = 425$

La DDS retourne 425 (vérification : $256 + 128 + 32 + 8 + 1 = 425$).

Variante avec attributs hérités : $N \rightarrow B N1$ avec la valeur partielle descendue en accumulateur ; la version ci-dessus est plus simple.

Exercice 5

On cherche le **nombre minimum de registres r** pour évaluer une expression. Principe (algorithme de Sethi-Ullman) : pour un opérateur binaire de fils gauche l et droit r (nombres de registres de chacun), on peut évaluer les deux opérandes dans l'un ou l'autre ordre :

- gauche d'abord : on garde le résultat gauche dans 1 registre pendant qu'on évalue la droite $\rightarrow \max(l, r+1)$;
- droite d'abord : $\max(r, l+1)$.
- On prend le meilleur des deux : **$\min(\max(l, r+1), \max(r, l+1)) = l+1$** si $l = r$, sinon $\max(l, r)$.

Notons $\text{ops}(l, r) = (\text{si } l = r \text{ alors } l+1 \text{ sinon } \max(l, r))$. Un identificateur demande 1 registre.

```
S -> E        { S.r := E.r }
E -> E1 + T    { E0.r := ops(E1.r, T.r) }
E -> E1 - T    { E0.r := ops(E1.r, T.r) }
E -> T        { E.r := T.r }
T -> T1 * F    { T0.r := ops(T1.r, F.r) }
T -> T1 / F    { T0.r := ops(T1.r, F.r) }
T -> F        { T.r := F.r }
F -> IDENT    { F.r := 1 }
F -> ( E )    { F.r := E.r }
```

r est synthétisé. **Vérification sur $a + b*c - d*(e+f)$** (arbre : $(a + b*c) - d*(e+f)$) :

Sous-expression	Calcul	r
$b*c$	$\text{ops}(1,1)$	2
$a + b*c$	$\text{ops}(1,2) = \max$	2
$e+f$	$\text{ops}(1,1)$	2
$(e+f)$	$F \rightarrow (E)$	2
$d*(e+f)$	$\text{ops}(1,2) = \max$	2
$(a+b*c) - d*(e+f)$	$\text{ops}(2,2) = 2+1$	3

On retrouve les 3 registres de la version 3 : on évalue d'abord la branche la plus gourmande ($b*c$ avant a ; $e+f$ avant d).

À retenir

Si on impose l'ordre gauche puis droite (formule $\max(l, r+1)$), on trouverait 4 registres pour cet exemple, donc pas le minimum : l'énoncé demande bien de choisir l'ordre d'évaluation des opérandes.

Remarque : pour le « - » ou le « / », évaluer la droite d'abord reste valable : les deux opérandes sont calculés dans des registres distincts avant d'effectuer l'opération.